Aula 23
Grafos

Algoritmos sobre Grafos
Algoritmos e Estruturas de Dados

Algoritmos

Sobre Grafos

Determinar

vértices alcançáveis

Determinar vértices alcançáveis

- Problema:
  - Dado um grafo dirigido G, e um vértice v, como determinar todos os vértices alcançáveis a partir de v?

Definição:
Um vértice v_2 diz-se alcançável a partir de outro vértice v_1 sse existir um caminho que parta de v_1 e chegue a v_2

Determinar vértices alcançaveis

- Problema:
  - Dado um grafo dirigido G, e um vértice v, como determinar todos os vértices alcançáveis a partir de v?

Definição:
Um vértice v_2 diz-se alcançável a partir de outro vértice v_1 sse existir um caminho que parta de v_1 e chegue a v_2

É possível responder a este problema usando uma procura em profundidade primeiro (Depth-first search)

Procura em Profundidade Primeiro

- Ideia:
  - Partindo do vértice inicial v

  1)Visito o vértice

  a) marco o vértice como visitado

  b) escolho um dos vértices adjacentes para visitar (que ainda não tenha sido visitado)

  i) se não houver vértices novos a visitar, volto para trás, e escolho outra alternativa

  ii) se houver vértices, visito o próximo (repete o processo a partir de 1)

  - Depois da procura terminar, todos os vértices marcados como visitados são alcançáveis a partir de v

Procura em Profundidade Primeiro

```
public class RecursiveDFS {
      private boolean[] visited;
      private int count;
      private DiGraph graph;

      public RecursiveDFS(DiGraph g)
      {
            this.graph = g;
            this.visited = new boolean[g.vCount()];
      }

      public void search(int startingV)
      {
            int vertices = this.graph.vCount();
            //initialize visited array to false
            for(int i = 0; i<vertices;i++)
            {
                  this.visited[i] = false;
            }
            //initialize number of visited vertices to 0
            this.count = 0;
            visit(startingV);
      }

      public boolean isReacheable(int destination) {return this.visited[destination];}

      public int count(){return this.count;}
```

Observação:
Esta procura é implementada como uma classe porque é um algoritmo que para além do Grafo recebido vai ter que guardar um estado adicional sobre o processo de procura, e queremos que essa informação se mantenha após a procura

Este método apenas inicializa e inicia o algoritmo de procura.

No final da procura, os campos visited e count vão ter a informação sobre quais e quantos são os vértices alcançáveis

Procura em Profundidade Primeiro

```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }
}
```

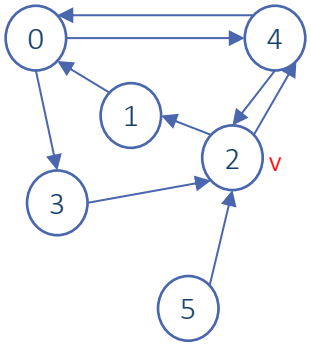
marcamos o nó como visitado

vamos visitar todos os nós adjacentes que ainda não foram visitados

Observação:
Esta é uma implementação recursiva da procura em profundidade primeiro.

O processo de andar para trás é conseguido através do modo de funcionamento de um processo recursivo.

Exemplo PPP



Vértice marcado como visitado

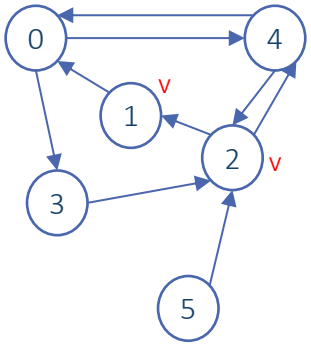
```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }
}
```

Call stack

search(2)
visit(2) → for [~~1~~,4]
visit(1)

Exemplo PPP



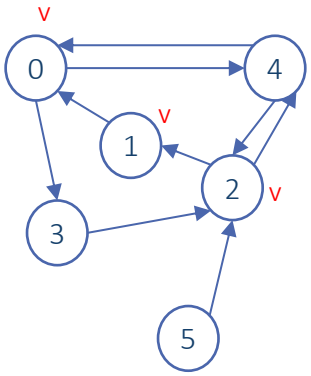

```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }
}
```

Call stack

search(2)
visit(2) → for [1,4]
visit(1) → for [0]
visit(0)

Exemplo PPP



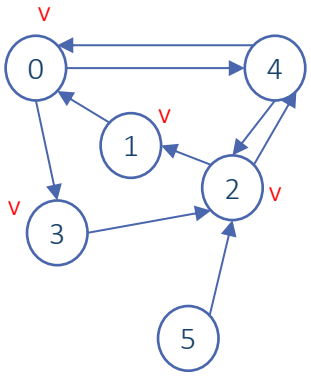

```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }
}
```

Call stack

search(2)
visit(2) → for [1,4]
visit(1) → for [0]
visit(0) → for [3,4]
visit(3)

Exemplo PPP



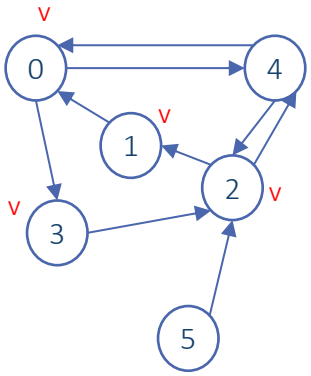

```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }
}
```

Call stack

search(2)
visit(2) → for [1,4]
visit(1) → for [0]
visit(0) → for [3,4]
visit(3) → for [2]

Exemplo PPP



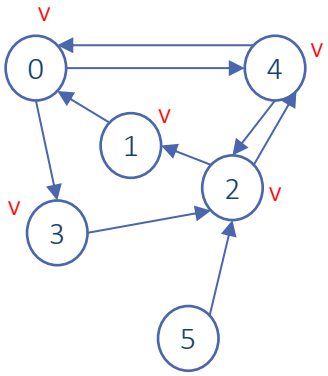

```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }
}
```

Call stack

search(2)
visit(2) → for [1,4]
visit(1) → for [0]
visit(0) → for [3,4]
visit(3) → for [2]
visit(4)

Exemplo PPP



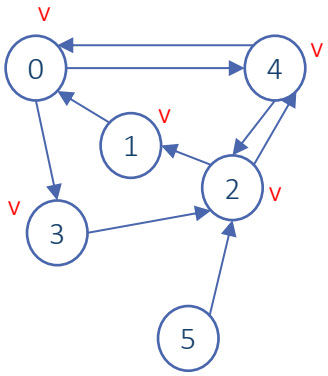

```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }
}
```

Call stack

search(2)
visit(2) → for [1,4]
visit(1) → for [0]
visit(0) → for [3,4]
visit(3) → for [2]
visit(4) → for [0,2]

Exemplo PPP



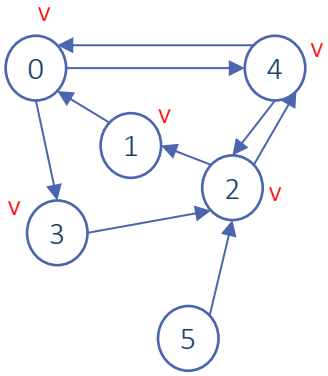

```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }
}
```

Call stack

search(2)
visit(2) → for [1,4]
visit(1) → for [0]
visit(0) → for [3,4]
visit(3) → for [2]
visit(4) → for [0,2]

Exemplo PPP




```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }
}
```

Call stack

search(2)
visit(2) → for [1,4]
visit(1) → for [0]
visit(0) → for [3,4]
visit(3) → for [2]
visit(4) → for [0,2]

PPP – versão não recursiva

- É possível implementar a procura em profundidade primeiro de forma não recursiva
  - Usando uma *Stack* para guardar os vértices que ainda não visitámos mas queremos visitar.

PPP – versão não recursiva

```
public class DepthFirstSearch {

      private boolean[] visited;
      private Stack<Integer> open;
      private int count;
      private DiGraph graph;

      public DepthFirstSearch(DiGraph g)
      {
            this.graph = g;
            this.visited = new boolean[g.vCount()];
      }
```

Observação:
Stack que vai conter os vértices que queremos visitar

Numa procura, esta estrutura é normalmente chamada de lista de abertos.

PPP – versão não recursiva

```
public void search(int startingV)
{
      Integer v;
      int vertices = this.graph.vCount();
      //initialize visited array to false
      for(int i = 0; i<vertices;i++)
      {
            this.visited[i] = false;
      }
      this.count = 0;
      this.open = new Stack<Integer>();
      this.open.push(startingV);

      while(!this.open.isEmpty())
      {
            v = this.open.pop();
            //if not visited, visit the node
            if(!this.visited[v]) visit(v);
      }
}
```

```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;

      for(int v2 : this.graph.adj(v))
      {
            this.open.push(v2);
      }
}
```

Observação:
Esta implementação é equivalente à implementação recursiva da Procura em Profundidade Primeiro. A *stack* aqui usada de forma explícita substitui a *call stack* da linguagem usada num procedimento recursivo.

Observação:
Um processo recursivo é caracterizado por uma fase de expansão, onde um conjunto de operações são adiadas, e uma fase de contração onde as operações adiadas são finalmente executadas.

Complexidade Temporal

- Algoritmo que determina todos os vértices alcançáveis a partir de v

- $T_{alcançável} = V + E$

onde **V** corresponde ao total de vertices do grafo,
e **E** corresponde ao total de arcos (edges) do grafo

- Se todos os vértices forem adjacentes entre si

$E = V^2$

$T_{alcançável} = V + V^2 \sim V^2$

Felizmente na prática isto não costuma acontecer

Complexidade Temporal

- Algoritmo que determina todos os vértices alcançáveis a partir de v

  - $T_{alcançável} = V + E$

  - Normalmente, grau(v) << V

    $E = k\ V$

    $T_{alcançável} = V + k\ V = (k+1)\ V$

    $T_{alcançável} = O(V)$

Na maior parte dos grafos, cada vértice está ligado apenas a um número reduzido de outros vértices

Nesta situação, o número de arcos depende linearmente do número de vértices. K é uma constante que corresponde ao grau médio dos vértices.

Detetar

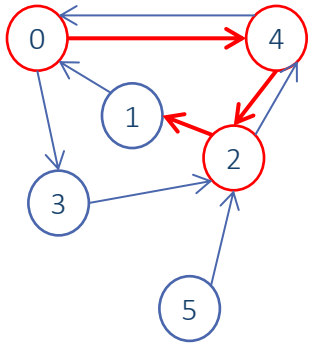
ciclos

Detetar ciclos

- Problema:
  - Dado um grafo dirigido G, como detetar se existe algum ciclo no grafo?

Detetar ciclos

- Ideia:
  - Fazer uma procura em profundidade primeiro (ou várias)
  - Guardar em memória os vértices do caminho atual da procura (a vermelho no diagrama)
  - Se encontrarmos um vértice que está no caminho atual, detetamos um ciclo



Implementação deteção ciclos

```
public class CycleDetector {
	private boolean[] visited;
	private boolean[] inCurrentPath;
	private DiGraph graph;
	private boolean hasCycle;

	public CycleDetector(DiGraph g)
	{
		this.graph = g;
		this.visited = new boolean[g.vCount()];
		this.inCurrentPath = new boolean[g.vCount()];
	}

	public void search()
	{
		int vertices = this.graph.vCount();
		this.hasCycle = false;
		//initialize array to false
		for(int i = 0; i<vertices;i++)
		{
			this.visited[i] = false;
			this.inCurrentPath[i] = false;
		}
		for(int i = 0; i < vertices; i++)
		{
			//start a new search for each vertex that has not been visited yet
			if(!this.visited[i]) visit(i);
			if(this.hasCycle) return;
		}
	}
}
```

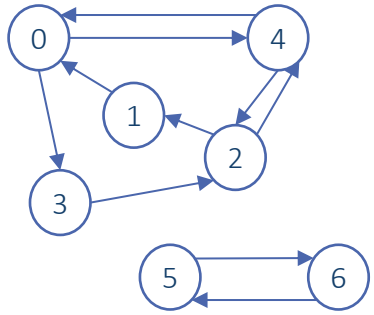
Observação:
Este array vai indicar quais os vértices que pertencem ao caminho que está a ser explorado a cada instante pela procura

Implementação deteção ciclos

```
public class CycleDetector {
	private boolean[] visited;
	private boolean[] inCurrentPath;
	private DiGraph graph;
	private boolean hasCycle;

	public CycleDetector(DiGraph g)
	{
		this.graph = g;
		this.visited = new boolean[g.vCount()];
		this.inCurrentPath = new boolean[g.vCount()];
	}

	public void search()
	{
		int vertices = this.graph.vCount();
		this.hasCycle = false;
		//initialize array to false
		for(int i = 0; i<vertices;i++)
		{
			this.visited[i] = false;
			this.inCurrentPath[i] = false;
		}
		for(int i = 0; i < vertices; i++)
		{
			//start a new search for each vertex that has not been visited yet
			if(!this.visited[i]) visit(i);
			if(this.hasCycle) return;
		}
	}
}
```



Observação:
Temos que fazer múltiplas procuras para o caso em que o Grafo não seja fortemente ligado, e existam vários componentes

Se o grafo for fortemente ligado, a primeira procura vai logo visitar todos os nós

Implementação deteção ciclos

```
private void visit(int v)
{
      this.inCurrentPath[v] = true;
      this.visited[v] = true;

      for(int adj : graph.adj(v))
      {
            //if a cycle was already detected we do not need to continue
            if(this.hasCycle) return;
            else if(!this.visited[adj]) visit(adj);
            //if a vertex was already visited, we need to check if that vertex already exists
            //in the current path (if so, we detected a cycle)
            else if(this.inCurrentPath[adj])
            {
                  this.hasCycle = true;
                  return;
            }
      }

      this.inCurrentPath[v] = false;
}

public boolean hasCycle()
{
      return this.hasCycle;
}
```

Memorizamos que este vértice pertence ao caminho atual que está a ser explorado pela procura

Quando voltamos para trás no processo recursivo da procura, o vértice v já não vai pertencer ao caminho que está a ser explorado atualmente

Complexidade temporal

- O Algoritmo de deteção de ciclos corresponde a
  - procura em profundidade primeiro
  - para cada um dos componentes fortemente ligados do Grafo

- No entanto, nunca visita um nó mais do que uma vez

- Pior caso, não existem ciclos, todos os vértices são visitados e todos os arcos são testados

  $T_{ciclos} = E + V$

- Quando existe ciclo, visitamos em média metade dos vértices e arcos, até encontrarmos um ciclo

  $T_{ciclos} = \sim \frac{E+V}{2}$